

Luniero Marketing Agent — Implementation Plan

Purpose: This document is a complete implementation plan for building the Luniero Marketing Agent. Copy this entire document into Claude Code and follow it step-by-step.

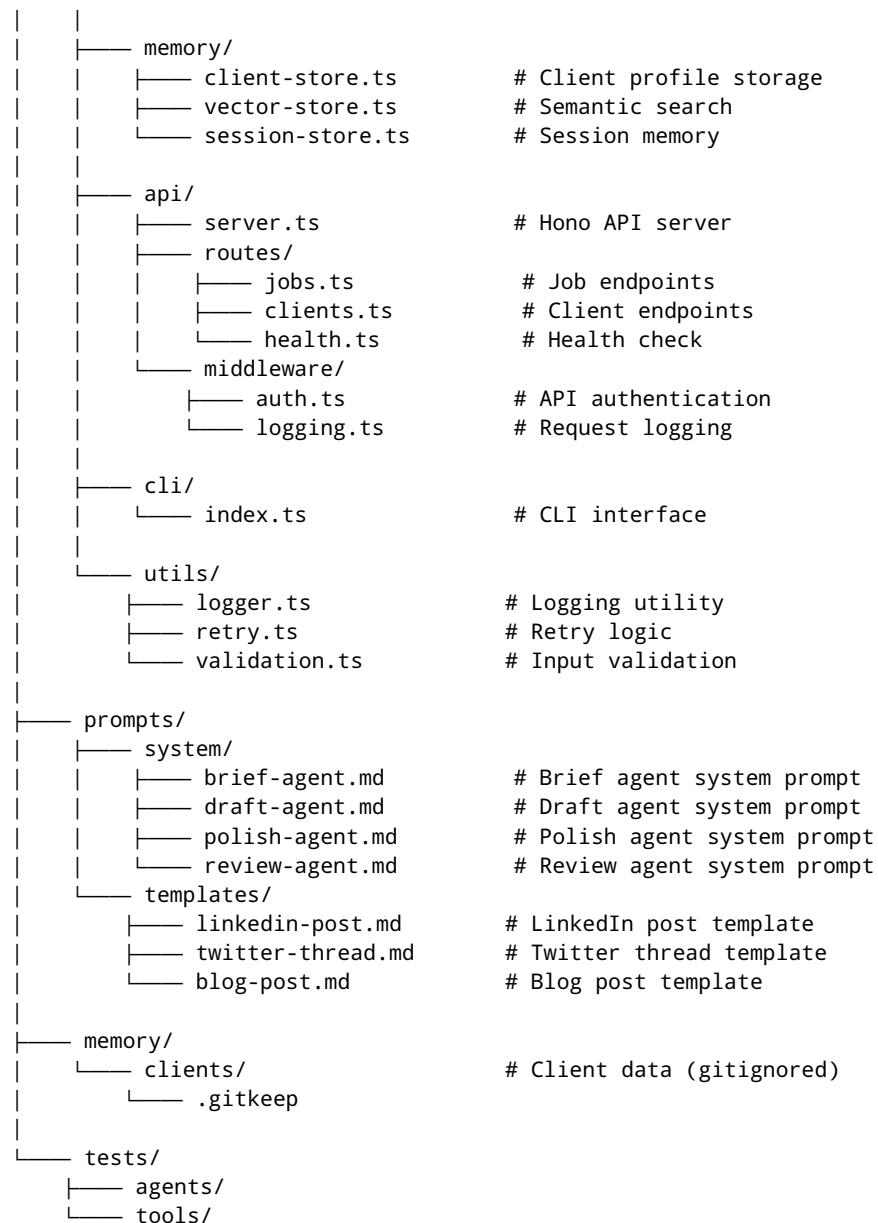
Project Overview

Build a multi-agent AI system for marketing agencies that: - Creates content across platforms - Generates reports - Tracks competitors - Manages client workflows - Uses distributed architecture with parallel processing

Tech Stack: - Runtime: Node.js + TypeScript - AI: Claude API (Anthropic) - Message Bus: Redis Streams - Database: Supabase (Postgres + pgvector) - API: Hono - UI: Next.js (later)

Project Structure

```
luniero-engine/
├── package.json
├── tsconfig.json
├── .env.example
├── .env
├──
├── src/
│   ├── index.ts                # Entry point
│   ├── config.ts              # Configuration loader
│   ├──
│   ├── core/
│   │   ├── message-bus.ts     # Redis Streams pub/sub
│   │   ├── state-store.ts     # Job state management
│   │   ├── event-types.ts     # Event schemas
│   │   └── agent-base.ts      # Base agent class
│   ├──
│   ├── agents/
│   │   ├── router.ts          # Routes incoming jobs
│   │   ├── supervisor.ts      # Health monitoring
│   │   ├── context-agent.ts   # Client context loading
│   │   ├── research-agent.ts  # Trend/competitor research
│   │   ├── brief-agent.ts     # Content brief creation
│   │   ├── draft-agent.ts     # First draft writing
│   │   ├── polish-agent.ts    # Voice/tone refinement
│   │   └── review-agent.ts    # Quality checking
│   ├──
│   ├── tools/
│   │   ├── index.ts           # Tool registry
│   │   ├── content-tools.ts   # Content creation tools
│   │   ├── platform-tools.ts  # Social platform tools
│   │   ├── analytics-tools.ts # Reporting tools
│   │   ├── research-tools.ts  # Research tools
│   │   └── client-tools.ts    # Client management tools
```



Phase 1: Foundation (Day 1)

Step 1.1: Project Setup

```
mkdir luniero-engine && cd luniero-engine
npm init -y
npm install typescript tsx @types/node --save-dev
npm install @anthropic-ai/sdk redis @supabase/supabase-js hono zod
dotenv
npx tsc --init
```

Create tsconfig.json:

```
{
  "compilerOptions": {
```

```

    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "node",
    "esModuleInterop": true,
    "strict": true,
    "outDir": "./dist",
    "rootDir": "./src",
    "declaration": true,
    "resolveJsonModule": true,
    "skipLibCheck": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist"]
}

```

Create .env.example:

```

ANTHROPIC_API_KEY=sk-ant-...
REDIS_URL=redis://localhost:6379
SUPABASE_URL=https://xxx.supabase.co
SUPABASE_KEY=eyJ...
PORT=3000
NODE_ENV=development

```

Step 1.2: Configuration (src/config.ts)

```

import { z } from 'zod';
import dotenv from 'dotenv';

dotenv.config();

const configSchema = z.object({
  anthropicApiKey: z.string(),
  redisUrl: z.string().default('redis://localhost:6379'),
  supabaseUrl: z.string(),
  supabaseKey: z.string(),
  port: z.number().default(3000),
  nodeEnv: z.enum(['development', 'production', 'test']).default('development'),
});

export const config = configSchema.parse({
  anthropicApiKey: process.env.ANTHROPIC_API_KEY,
  redisUrl: process.env.REDIS_URL,
  supabaseUrl: process.env.SUPABASE_URL,
  supabaseKey: process.env.SUPABASE_KEY,
  port: parseInt(process.env.PORT || '3000'),
  nodeEnv: process.env.NODE_ENV,
});

```

Step 1.3: Logger (src/utils/logger.ts)

```

type LogLevel = 'debug' | 'info' | 'warn' | 'error';

interface LogEntry {
  timestamp: string;
  level: LogLevel;
  agent?: string;
}

```

```

    jobId?: string;
    message: string;
    data?: any;
  }

  class Logger {
    private formatEntry(entry: LogEntry): string {
      const prefix = entry.agent ? `[${entry.agent}]` : '';
      const jobPrefix = entry.jobId ? `[${entry.jobId}]` : '';
      return `${entry.timestamp} ${entry.level.toUpperCase()}
${prefix}${jobPrefix} ${entry.message}`;
    }

    private log(level: LogLevel, message: string, data?: any,
context?: { agent?: string; jobId?: string }) {
      const entry: LogEntry = {
        timestamp: new Date().toISOString(),
        level,
        message,
        data,
        ...context,
      };

      console.log(this.formatEntry(entry));
      if (data) console.log(JSON.stringify(data, null, 2));
    }

    debug(message: string, data?: any, context?: { agent?: string;
jobId?: string }) {
      this.log('debug', message, data, context);
    }

    info(message: string, data?: any, context?: { agent?: string;
jobId?: string }) {
      this.log('info', message, data, context);
    }

    warn(message: string, data?: any, context?: { agent?: string;
jobId?: string }) {
      this.log('warn', message, data, context);
    }

    error(message: string, data?: any, context?: { agent?: string;
jobId?: string }) {
      this.log('error', message, data, context);
    }
  }

  export const logger = new Logger();

```

Step 1.4: Event Types (src/core/event-types.ts)

```

import { z } from 'zod';

// Base event schema
export const baseEventSchema = z.object({
  eventId: z.string().uuid(),
  jobId: z.string(),
  timestamp: z.string().datetime(),

```

```

    sourceAgent: z.string(),
    targetAgents: z.array(z.string()).optional(),
    clientId: z.string(),
    traceId: z.string().uuid(),
    parentEventId: z.string().uuid().optional(),
  });

  // Event types
  export const eventTypes = {
    JOB_CREATED: 'job.created',
    CONTEXT_LOADED: 'context.loaded',
    RESEARCH_DONE: 'research.done',
    BRIEF_READY: 'brief.ready',
    BRIEF_APPROVED: 'brief.approved',
    DRAFT_READY: 'draft.ready',
    POLISH_DONE: 'polish.done',
    REVIEW_PASSED: 'review.passed',
    REVIEW_FAILED: 'review.failed',
    REVISION_REQUESTED: 'revision.requested',
    JOB_COMPLETE: 'job.complete',
    AGENT_ERROR: 'agent.error',
  } as const;

  export type EventType = keyof typeof eventTypes[keyof typeof eventTypes];

  // Payload schemas for each event type
  export const jobCreatedPayload = z.object({
    type: z.enum(['social_post', 'blog_post', 'report', 'campaign']),
    platform: z.enum(['linkedin', 'twitter', 'instagram', 'facebook',
'tiktok', 'all']).optional(),
    topic: z.string(),
    instructions: z.string().optional(),
  });

  export const contextPayload = z.object({
    brandVoice: z.object({
      tone: z.string(),
      avoid: z.array(z.string()),
      examples: z.array(z.string()),
    }),
    contentPillars: z.array(z.string()),
    preferences: z.record(z.string(), z.any()),
  });

  export const briefPayload = z.object({
    title: z.string(),
    type: z.string(),
    targetAudience: z.string(),
    keyMessages: z.array(z.string()),
    structure: z.array(z.object({
      section: z.string(),
      notes: z.string(),
    })),
    wordCount: z.number(),
    tone: z.string(),
    seo: z.object({
      primaryKeyword: z.string(),
      secondaryKeywords: z.array(z.string()),
    }).optional(),
  });

```

```

export const draftPayload = z.object({
  content: z.string(),
  wordCount: z.number(),
  format: z.string(),
});

export const reviewPayload = z.object({
  status: z.enum(['approved', 'needs_revision',
'needs_human_review']),
  score: z.number().min(0).max(100),
  checks: z.record(z.string(), z.object({
    pass: z.boolean(),
    notes: z.string().optional(),
  })),
  revisionRequests: z.array(z.string()).optional(),
});

// Full event schema
export const agentEventSchema = baseEventSchema.extend({
  eventType: z.string(),
  payload: z.any(),
});

export type AgentEvent = z.infer<typeof agentEventSchema>;

```

Step 1.5: Message Bus (src/core/message-bus.ts)

```

import { createClient, RedisClientType } from 'redis';
import { config } from '../config';
import { logger } from '../utils/logger';
import { AgentEvent, EventType } from '../event-types';
import { v4 as uuidv4 } from 'uuid';

export class MessageBus {
  private client: RedisClientType;
  private subscribers: Map<string, ((event: AgentEvent) =>
Promise<void>)[> = new Map();
  private consumerGroup = 'luniero-agents';
  private streamName = 'agent-events';

  async connect() {
    this.client = createClient({ url: config.redisUrl });
    await this.client.connect();

    // Create stream and consumer group if they don't exist
    try {
      await this.client.xGroupCreate(this.streamName,
this.consumerGroup, '0', { MKSTREAM: true });
    } catch (err: any) {
      if (!err.message.includes('BUSYGROUP')) throw err;
    }

    logger.info('Message bus connected');
  }

  async publish(event: Omit<AgentEvent, 'eventId' | 'timestamp'>):
Promise<string> {
    const fullEvent: AgentEvent = {

```

```

        ...event,
        eventId: uuidv4(),
        timestamp: new Date().toISOString(),
    });

    await this.client.xAdd(this.streamName, '*', {
        data: JSON.stringify(fullEvent),
    });

    logger.debug(`Published event: ${event.eventType}`, { jobId:
event.jobId });
    return fullEvent.eventId;
}

subscribe(eventType: EventType | EventType[], handler: (event:
AgentEvent) => Promise<void>) {
    const types = Array.isArray(eventType) ? eventType :
[eventType];

    for (const type of types) {
        if (!this.subscribers.has(type)) {
            this.subscribers.set(type, []);
        }
        this.subscribers.get(type)!.push(handler);
    }
}

async startConsuming(consumerName: string) {
    logger.info(`Consumer ${consumerName} starting...`);

    while (true) {
        try {
            const results = await this.client.xReadGroup(
                this.consumerGroup,
                consumerName,
                { key: this.streamName, id: '>' },
                { COUNT: 10, BLOCK: 5000 }
            );

            if (!results) continue;

            for (const result of results) {
                for (const message of result.messages) {
                    const event: AgentEvent =
JSON.parse(message.message.data);

                    const handlers = this.subscribers.get(event.eventType)
|| [];

                    for (const handler of handlers) {
                        try {
                            await handler(event);
                        } catch (err) {
                            logger.error(`Handler error for ${event.eventType}`,
err);
                        }
                    }

                    // Acknowledge message
                    await this.client.xAck(this.streamName,
this.consumerGroup, message.id);

```

```

        }
      }
    } catch (err) {
      logger.error('Consumer error', err);
      await new Promise(r => setTimeout(r, 1000));
    }
  }
}

async disconnect() {
  await this.client.quit();
}
}

export const messageBus = new MessageBus();

```

Step 1.6: State Store (src/core/state-store.ts)

```

import { createClient } from '@supabase/supabase-js';
import { config } from '../config';
import { logger } from '../utils/logger';

export type JobStatus =
  | 'received'
  | 'researching'
  | 'context_loading'
  | 'briefing'
  | 'brief_pending_approval'
  | 'drafting'
  | 'polishing'
  | 'reviewing'
  | 'revision'
  | 'human_review'
  | 'complete'
  | 'failed';

export interface Job {
  id: string;
  clientId: string;
  type: string;
  status: JobStatus;
  input: any;
  context?: any;
  research?: any;
  brief?: any;
  draft?: any;
  polishedDraft?: any;
  review?: any;
  output?: any;
  iteration: number;
  maxIterations: number;
  createdAt: string;
  updatedAt: string;
  completedAt?: string;
  error?: string;
}

class StateStore {
  private supabase;

```



```

    constructor() {
        this.supabase = createClient(config.supabaseUrl,
config.supabaseKey);
    }

    async createJob(job: Omit<Job, 'createdAt' | 'updatedAt' |
'iteration'>): Promise<Job> {
        const now = new Date().toISOString();
        const fullJob: Job = {
            ...job,
            iteration: 0,
            maxIterations: job.maxIterations || 3,
            createdAt: now,
            updatedAt: now,
        };

        const { data, error } = await this.supabase
            .from('jobs')
            .insert(fullJob)
            .select()
            .single();

        if (error) throw error;
        logger.info(`Job created: ${job.id}`, { jobId: job.id });
        return data;
    }

    async getJob(jobId: string): Promise<Job | null> {
        const { data, error } = await this.supabase
            .from('jobs')
            .select('*')
            .eq('id', jobId)
            .single();

        if (error) return null;
        return data;
    }

    async updateJob(jobId: string, updates: Partial<Job>):
Promise<Job> {
        const { data, error } = await this.supabase
            .from('jobs')
            .update({ ...updates, updatedAt: new Date().toISOString() })
            .eq('id', jobId)
            .select()
            .single();

        if (error) throw error;
        logger.debug(`Job updated: ${jobId}`, { status: updates.status
});

        return data;
    }

    async incrementIteration(jobId: string): Promise<number> {
        const job = await this.getJob(jobId);
        if (!job) throw new Error(`Job not found: ${jobId}`);

        const newIteration = job.iteration + 1;
        await this.updateJob(jobId, { iteration: newIteration });
    }

```

```

        return newIteration;
    }

    async getJobsByClient(clientId: string, limit = 20):
Promise<Job[]> {
        const { data, error } = await this.supabase
            .from('jobs')
            .select('*')
            .eq('clientId', clientId)
            .order('createdAt', { ascending: false })
            .limit(limit);

        if (error) throw error;
        return data || [];
    }
}

export const stateStore = new StateStore();

```

Step 1.7: Base Agent Class (src/core/agent-base.ts)

```

import Anthropic from '@anthropic-ai/sdk';
import { config } from '../config';
import { messageBus } from '../message-bus';
import { stateStore, Job } from '../state-store';
import { logger } from '../utils/logger';
import { AgentEvent, EventType } from '../event-types';
import { readFileSync } from 'fs';
import { join } from 'path';

export abstract class BaseAgent {
    protected name: string;
    protected anthropic: Anthropic;
    protected systemPrompt: string;

    constructor(name: string) {
        this.name = name;
        this.anthropic = new Anthropic({ apiKey: config.anthropicApiKey });

        this.systemPrompt = this.loadSystemPrompt();
    }

    private loadSystemPrompt(): string {
        try {
            const path = join(__dirname, '../prompts/system',
`${this.name}.md`);
            return readFileSync(path, 'utf-8');
        } catch {
            return this.getDefaultSystemPrompt();
        }
    }

    protected abstract getDefaultSystemPrompt(): string;
    protected abstract getSubscribedEvents(): EventType[];
    protected abstract handleEvent(event: AgentEvent, job: Job):
Promise<void>;

    async start() {
        const events = this.getSubscribedEvents();
    }
}

```

```

        messageBus.subscribe(events, async (event) => {
            const job = await stateStore.getJob(event.jobId);
            if (!job) {
                logger.error(`Job not found: ${event.jobId}`, { agent:
this.name });
                return;
            }

            try {
                logger.info(`Processing ${event.eventType}`, { agent:
this.name, jobId: event.jobId });
                await this.handleEvent(event, job);
            } catch (error) {
                logger.error(`Error processing event`, error, { agent:
this.name, jobId: event.jobId });
                await this.publishError(event, error);
            }
        });

        logger.info(`${this.name} started, listening for:
${events.join(', ')}');
    }

    protected async callLLM(messages: { role: 'user' | 'assistant';
content: string }[], options?: {
        maxTokens?: number;
        temperature?: number;
    }): Promise<string> {
        const response = await this.anthropic.messages.create({
            model: 'claude-sonnet-4-20250514',
            max_tokens: options?.maxTokens || 4096,
            temperature: options?.temperature || 0.7,
            system: this.systemPrompt,
            messages,
        });

        const textBlock = response.content.find(block => block.type ===
'text');

        return textBlock?.text || '';
    }

    protected async publish(
        eventType: EventType,
        jobId: string,
        clientId: string,
        traceId: string,
        payload: any,
        parentEventId?: string
    ) {
        await messageBus.publish({
            eventType,
            jobId,
            clientId,
            traceId,
            sourceAgent: this.name,
            payload,
            parentEventId,
        });
    }

```

```

protected async publishError(event: AgentEvent, error: any) {
  await messageBus.publish({
    eventType: 'agent.error',
    jobId: event.jobId,
    clientId: event.clientId,
    traceId: event.traceId,
    sourceAgent: this.name,
    payload: {
      error: error.message,
      stack: error.stack,
      originalEvent: event.eventType,
    },
    parentEventId: event.eventId,
  });
}
}

```

Phase 2: Agents (Day 1-2)

Step 2.1: Router Agent (src/agents/router.ts)

```

import { v4 as uuidv4 } from 'uuid';
import { messageBus } from '../core/message-bus';
import { stateStore } from '../core/state-store';
import { eventTypes, AgentEvent } from '../core/event-types';
import { logger } from '../utils/logger';

interface JobRequest {
  clientId: string;
  type: 'social_post' | 'blog_post' | 'report' | 'campaign';
  platform?: string;
  topic: string;
  instructions?: string;
}

class RouterAgent {
  name = 'router';

  async start() {
    // Router listens for job.complete to finalize
    messageBus.subscribe([eventTypes.REVIEW_PASSED], async (event) => {
      await this.handleReviewPassed(event);
    });

    logger.info('Router agent started');
  }

  async createJob(request: JobRequest): Promise<string> {
    const jobId = uuidv4();
    const traceId = uuidv4();

    // Create job in state store
    await stateStore.createJob({
      id: jobId,
      clientId: request.clientId,
      type: request.type,

```

```

        status: 'received',
        input: request,
        maxIterations: 3,
    });

    // Publish job.created event
    await messageBus.publish({
        eventType: eventTypes.JOB_CREATED,
        jobId,
        clientId: request.clientId,
        traceId,
        sourceAgent: this.name,
        payload: request,
    });

    logger.info(`Job created and routed: ${jobId}`, { type:
request.type, client: request.clientId });
    return jobId;
}

private async handleReviewPassed(event: AgentEvent) {
    const job = await stateStore.getJob(event.jobId);
    if (!job) return;

    // Update job to complete
    await stateStore.updateJob(event.jobId, {
        status: 'complete',
        output: job.polishedDraft || job.draft,
        completedAt: new Date().toISOString(),
    });

    // Publish job.complete
    await messageBus.publish({
        eventType: eventTypes.JOB_COMPLETE,
        jobId: event.jobId,
        clientId: event.clientId,
        traceId: event.traceId,
        sourceAgent: this.name,
        payload: {
            output: job.polishedDraft || job.draft,
            stats: {
                iterations: job.iteration,
                reviewScore: event.payload.score,
            },
        },
    });

    logger.info(`Job completed: ${event.jobId}`);
}

export const routerAgent = new RouterAgent();

```

Step 2.2: Context Agent (src/agents/context-agent.ts)

```

import { BaseAgent } from '../core/agent-base';
import { stateStore } from '../core/state-store';
import { eventTypes, AgentEvent } from '../core/event-types';
import { clientStore } from '../memory/client-store';

```

```

class ContextAgent extends BaseAgent {
  constructor() {
    super('context-agent');
  }

  protected getDefaultSystemPrompt(): string {
    return `You are the Context Agent. Your job is to load and
organize client context for content creation.`;
  }

  protected getSubscribedEvents() {
    return [eventTypes.JOB_CREATED];
  }

  protected async handleEvent(event: AgentEvent, job: any) {
    // Load client profile
    const clientProfile = await
clientStore.getProfile(event.clientId);
    const brandVoice = await
clientStore.getBrandVoice(event.clientId);
    const contentPillars = await
clientStore.getContentPillars(event.clientId);
    const recentFeedback = await
clientStore.getRecentFeedback(event.clientId, 5);

    const context = {
      profile: clientProfile,
      brandVoice,
      contentPillars,
      recentFeedback,
      preferences: clientProfile?.preferences || {},
    };

    // Update job with context
    await stateStore.updateJob(event.jobId, {
      status: 'context_loading',
      context,
    });

    // Publish context.loaded
    await this.publish(
      eventTypes.CONTEXT_LOADED,
      event.jobId,
      event.clientId,
      event.traceId,
      context,
      event.eventId
    );
  }
}

export const contextAgent = new ContextAgent();

```

Step 2.3: Brief Agent (src/agents/brief-agent.ts)

```

import { BaseAgent } from '../core/agent-base';
import { stateStore } from '../core/state-store';
import { eventTypes, AgentEvent } from '../core/event-types';

```

```

import { logger } from '../utils/logger';

class BriefAgent extends BaseAgent {
  constructor() {
    super('brief-agent');
  }

  protected getDefaultSystemPrompt(): string {
    return `You are the Brief Agent for a marketing agency. You
create detailed content briefs.

Your job:
1. Analyze the content request and client context
2. Create a structured brief that will guide the Draft Agent
3. Include all necessary details: structure, key messages, tone,
word count

Output Format (JSON):
{
  "title": "Proposed title",
  "type": "linkedin_post | twitter_thread | blog_post | etc",
  "targetAudience": "Who this is for",
  "keyMessages": ["Message 1", "Message 2"],
  "structure": [
    {"section": "hook", "notes": "What to include"},
    {"section": "body", "notes": "Main points"},
    {"section": "cta", "notes": "Call to action"}
  ],
  "wordCount": 150,
  "tone": "Professional but friendly",
  "platform": "linkedin",
  "hashtags": ["#relevant", "#tags"],
  "seo": {
    "primaryKeyword": "main keyword",
    "secondaryKeywords": ["other", "keywords"]
  }
}

Be specific. The Draft Agent will follow your brief exactly.`;
  }

  protected getSubscribedEvents() {
    return [eventTypes.CONTEXT_LOADED, eventTypes.RESEARCH_DONE];
  }

  protected async handleEvent(event: AgentEvent, job: any) {
    // Wait for both context and research (if research agent is
running)
    // For MVP, proceed after context is loaded

    await stateStore.updateJob(event.jobId, { status: 'briefing' });

    const prompt = this.buildPrompt(job);
    const response = await this.callLLM([
      { role: 'user', content:
prompt }
    ]);

    // Parse JSON from response
    const brief = this.parseJsonResponse(response);

    // Update job with brief

```

```

        await stateStore.updateJob(event.jobId, {
            brief,
            status: 'brief_pending_approval', // Or 'drafting' if auto-
approve
        });

        // Publish brief.ready
        await this.publish(
            eventTypes.BRIEF_READY,
            event.jobId,
            event.clientId,
            event.traceId,
            brief,
            event.eventId
        );

        logger.info(`Brief created for job ${event.jobId}`, { title:
brief.title });
    }

    private buildPrompt(job: any): string {
        return `Create a content brief for the following request:

        **Request:**
        - Type: ${job.input.type}
        - Topic: ${job.input.topic}
        - Platform: ${job.input.platform || 'general'}
        ${job.input.instructions ? '- Instructions:
${job.input.instructions}` : ''}

        **Client Context:**
        - Brand Voice: ${JSON.stringify(job.context?.brandVoice || {})}
        - Content Pillars: ${JSON.stringify(job.context?.contentPillars ||
[])}
        - Preferences: ${JSON.stringify(job.context?.preferences || {})}

        Create a detailed brief in JSON format.`;
    }

    private parseJsonResponse(response: string): any {
        try {
            // Extract JSON from response (might be wrapped in markdown)
            const jsonMatch = response.match(/\s{[\s\S]*\s}/);
            if (jsonMatch) {
                return JSON.parse(jsonMatch[0]);
            }
            throw new Error('No JSON found in response');
        } catch (error) {
            logger.error('Failed to parse brief response', { response });
            throw error;
        }
    }
}

export const briefAgent = new BriefAgent();

```

Step 2.4: Draft Agent (src/agents/draft-agent.ts)

```
import { BaseAgent } from '../core/agent-base';
```



```

import { stateStore } from '../core/state-store';
import { eventTypes, AgentEvent } from '../core/event-types';
import { logger } from '../utils/logger';

class DraftAgent extends BaseAgent {
  constructor() {
    super('draft-agent');
  }

  protected getDefaultSystemPrompt(): string {
    return `You are the Draft Agent for a marketing agency. You
write compelling first drafts.

Your job:
1. Follow the brief EXACTLY
2. Match the client's brand voice
3. Hit the word count target (within 10%)
4. Write engaging content that achieves the goal

Rules:
- Hook the reader in the first sentence
- Every section should flow into the next
- Include keywords naturally (never stuff)
- End with a clear call-to-action
- Match the specified tone

Output the content only. No explanations or meta-commentary.`;
  }

  protected getSubscribedEvents() {
    return [eventTypes.BRIEF_READY, eventTypes.BRIEF_APPROVED,
eventTypes.REVISION_REQUESTED];
  }

  protected async handleEvent(event: AgentEvent, job: any) {
    await stateStore.updateJob(event.jobId, { status: 'drafting' });

    // Handle revision requests
    let revisionContext = '';
    if (event.eventType === eventTypes.REVISION_REQUESTED) {
      const revision = event.payload;
      revisionContext = `

**REVISION REQUESTED:**
Previous draft had these issues:
${revision.issues.map((i: string) => `- ${i}`).join('\n')}

Please address these issues in your new draft.`;
    }

    const prompt = this.buildPrompt(job, revisionContext);
    const response = await this.callLLM([ { role: 'user', content:
prompt } ], {
      maxTokens: 8192,
      temperature: 0.8,
    });

    // Update job with draft
    await stateStore.updateJob(event.jobId, {
      draft: {

```

```

        content: response,
        wordCount: response.split(/\s+/).length,
        iteration: job.iteration,
    },
});

// Publish draft.ready
await this.publish(
    eventTypes.DRAFT_READY,
    event.jobId,
    event.clientId,
    event.traceId,
    { content: response },
    event.eventId
);

logger.info(`Draft created for job ${event.jobId}`, {
    wordCount: response.split(/\s+/).length,
    iteration: job.iteration
});
}

private buildPrompt(job: any, revisionContext: string): string {
    const brief = job.brief;

    return `Write content based on this brief:

    **Brief:**
    - Title: ${brief.title}
    - Type: ${brief.type}
    - Platform: ${brief.platform || 'general'}
    - Target Audience: ${brief.targetAudience}
    - Word Count: ${brief.wordCount}
    - Tone: ${brief.tone}

    **Key Messages:**
    ${brief.keyMessages.map((m: string) => `- ${m}`).join('\n')}

    **Structure:**
    ${brief.structure.map((s: any) => `- ${s.section}: ${s.notes}`).join('\n')}

    **Brand Voice:**
    ${JSON.stringify(job.context?.brandVoice || {})}
    ${revisionContext}

    Write the content now. Output ONLY the final content, nothing
else.`;
}

export const draftAgent = new DraftAgent();

```

Step 2.5: Polish Agent (src/agents/polish-agent.ts)

```

import { BaseAgent } from '../core/agent-base';
import { stateStore } from '../core/state-store';
import { eventTypes, AgentEvent } from '../core/event-types';
import { logger } from '../utils/logger';

```

```

class PolishAgent extends BaseAgent {
  constructor() {
    super('polish-agent');
  }

  protected getDefaultSystemPrompt(): string {
    return `You are the Polish Agent for a marketing agency. You
refine drafts to perfection.

Your job:
1. Ensure brand voice consistency
2. Improve readability and flow
3. Strengthen hooks and CTAs
4. Fix awkward phrasing
5. Verify the content matches the brief

Focus on:
- Voice consistency: Does it sound like the brand?
- Readability: Short sentences, clear structure
- Engagement: Strong hooks, smooth transitions
- Clarity: Remove jargon, simplify complexity
- Impact: Powerful opening and closing

Keep the original meaning. Improve the delivery.

Output ONLY the polished content. No explanations.`;
  }

  protected getSubscribedEvents() {
    return [eventTypes.DRAFT_READY];
  }

  protected async handleEvent(event: AgentEvent, job: any) {
    await stateStore.updateJob(event.jobId, { status: 'polishing'
});

    const prompt = this.buildPrompt(job);
    const response = await this.callLLM([
      { role: 'user', content:
prompt }], {
      temperature: 0.5, // Lower temperature for more consistent
      polish
    });

    // Update job with polished draft
    await stateStore.updateJob(event.jobId, {
      polishedDraft: {
        content: response,
        wordCount: response.split(/\s+/).length,
      },
    });

    // Publish polish.done
    await this.publish(
      eventTypes.POLISH_DONE,
      event.jobId,
      event.clientId,
      event.traceId,
      { content: response },
      event.eventId
    );
  }
}

```

```

    );

    logger.info(`Draft polished for job ${event.jobId}`);
  }

  private buildPrompt(job: any): string {
    return `Polish this draft to match the brand voice and maximize
engagement:

**Draft:**
${job.draft.content}

**Brief:**
- Type: ${job.brief.type}
- Tone: ${job.brief.tone}
- Platform: ${job.brief.platform || 'general'}

**Brand Voice:**
- Tone: ${job.context?.brandVoice?.tone || 'professional'}
- Avoid: ${JSON.stringify(job.context?.brandVoice?.avoid || [])}
- Examples: ${JSON.stringify(job.context?.brandVoice?.examples ||
[])}

    Polish the content while maintaining the core message. Output ONLY
the polished content.`;
  }
}

export const polishAgent = new PolishAgent();

```

Step 2.6: Review Agent (src/agents/review-agent.ts)

```

import { BaseAgent } from '../core/agent-base';
import { stateStore } from '../core/state-store';
import { eventTypes, AgentEvent } from '../core/event-types';
import { logger } from '../utils/logger';

class ReviewAgent extends BaseAgent {
  constructor() {
    super('review-agent');
  }

  protected getDefaultSystemPrompt(): string {
    return `You are the Review Agent for a marketing agency. You
ensure quality before delivery.

Your job:
1. Verify the content matches the brief
2. Check brand voice consistency
3. Validate word count and structure
4. Identify any issues

Output Format (JSON):
{
  "status": "approved" | "needs_revision" | "needs_human_review",
  "score": 0-100,
  "checks": {
    "brief_compliance": {"pass": true/false, "notes": "..."},
    "word_count": {"pass": true/false, "actual": N, "target": N},

```

```

        "brand_voice": {"pass": true/false, "confidence": 0.0-1.0},
        "structure": {"pass": true/false, "notes": "..."},
        "cta_present": {"pass": true/false},
        "hook_strength": {"pass": true/false, "notes": "..."}
    },
    "issues": ["Issue 1", "Issue 2"],
    "strengths": ["Strength 1", "Strength 2"]
}

Score Guidelines:
- 90-100: Excellent, ready to publish
- 80-89: Good, minor improvements possible
- 70-79: Acceptable, could be better
- Below 70: Needs revision`
}

protected getSubscribedEvents() {
    return [eventTypes.POLISH_DONE];
}

protected async handleEvent(event: AgentEvent, job: any) {
    await stateStore.updateJob(event.jobId, { status: 'reviewing'
});

    const prompt = this.buildPrompt(job);
    const response = await this.callLLM([ { role: 'user', content:
prompt } ], {
        temperature: 0.3, // Low temperature for consistent evaluation
    });

    const review = this.parseJsonResponse(response);

    // Update job with review
    await stateStore.updateJob(event.jobId, { review });

    // Determine next action based on review
    if (review.status === 'approved') {
        await this.publish(
            eventTypes.REVIEW_PASSED,
            event.jobId,
            event.clientId,
            event.traceId,
            review,
            event.eventId
        );
        logger.info(`Review passed for job ${event.jobId}`, { score:
review.score });
    } else if (review.status === 'needs_revision') {
        // Check iteration count
        const iteration = await
stateStore.incrementIteration(event.jobId);
        const updatedJob = await stateStore.getJob(event.jobId);

        if (iteration >= (updatedJob?.maxIterations || 3)) {
            // Max iterations reached, send to human review
            await stateStore.updateJob(event.jobId, { status:
'human_review' });
            review.status = 'needs_human_review';
            review.notes = 'Max iterations reached';
        } else {

```

```

        // Request revision
        await this.publish(
            eventTypes.REVISION_REQUESTED,
            event.jobId,
            event.clientId,
            event.traceId,
            { issues: review.issues, iteration },
            event.eventId
        );
        logger.info(`Revision requested for job ${event.jobId}`, {
            iteration, issues: review.issues });
    }
}

    if (review.status === 'needs_human_review') {
        await stateStore.updateJob(event.jobId, { status:
'human_review' });
        logger.info(`Human review required for job ${event.jobId}`);
    }
}

    private buildPrompt(job: any): string {
        return `Review this content against the brief and brand
guidelines:

        **Content:**
        ${job.polishedDraft?.content || job.draft?.content}

        **Brief:**
        ${JSON.stringify(job.brief, null, 2)}

        **Brand Voice:**
        ${JSON.stringify(job.context?.brandVoice || {}, null, 2)}

        Evaluate and output your review as JSON.`;
    }

    private parseJsonResponse(response: string): any {
        try {
            const jsonMatch = response.match(/\[{\s\S}*\/);
            if (jsonMatch) {
                return JSON.parse(jsonMatch[0]);
            }
            throw new Error('No JSON found');
        } catch {
            return {
                status: 'needs_human_review',
                score: 0,
                checks: {},
                issues: ['Failed to parse review response'],
                strengths: [],
            };
        }
    }
}

export const reviewAgent = new ReviewAgent();

```

Phase 3: Memory & Tools (Day 2)

Step 3.1: Client Store (src/memory/client-store.ts)

```
import { createClient } from '@supabase/supabase-js';
import { config } from '../config';
import { readFileSync, writeFileSync, existsSync, mkdirSync } from
'fs';

import { join } from 'path';

interface ClientProfile {
  id: string;
  name: string;
  industry: string;
  description: string;
  goals: string[];
  platforms: { platform: string; handle: string; frequency: string }
  [];

  contacts: { name: string; role: string; email: string }[];
  preferences: Record<string, any>;
}

interface BrandVoice {
  tone: string;
  avoid: string[];
  examples: string[];
  vocabulary: string[];
}

class ClientStore {
  private supabase;
  private localPath = join(__dirname, '../../memory/clients');

  constructor() {
    this.supabase = createClient(config.supabaseUrl,
config.supabaseKey);
    if (!existsSync(this.localPath)) {
      mkdirSync(this.localPath, { recursive: true });
    }
  }

  async getProfile(clientId: string): Promise<ClientProfile | null>
{
  // Try local first (for development)
  const localPath = join(this.localPath, clientId,
'profile.json');
  if (existsSync(localPath)) {
    return JSON.parse(readFileSync(localPath, 'utf-8'));
  }

  // Then try Supabase
  const { data, error } = await this.supabase
    .from('client_profiles')
    .select('*')
    .eq('id', clientId)
    .single();

  if (error) return null;
  return data;
}
```

```

    }

    async saveProfile(clientId: string, profile: ClientProfile) {
      // Save locally
      const clientPath = join(this.localPath, clientId);
      if (!existsSync(clientPath)) {
        mkdirSync(clientPath, { recursive: true });
      }
      writeFileSync(join(clientPath, 'profile.json'),
        JSON.stringify(profile, null, 2));

      // Save to Supabase
      await this.supabase.from('client_profiles').upsert(profile);
    }

    async getBrandVoice(clientId: string): Promise<BrandVoice | null>
    {
      const localPath = join(this.localPath, clientId,
        'brand_voice.json');
      if (existsSync(localPath)) {
        return JSON.parse(readFileSync(localPath, 'utf-8'));
      }

      const { data } = await this.supabase
        .from('brand_voices')
        .select('*')
        .eq('client_id', clientId)
        .single();

      return data;
    }

    async getContentPillars(clientId: string): Promise<string[]> {
      const profile = await this.getProfile(clientId);
      return profile?.preferences?.contentPillars || [];
    }

    async getRecentFeedback(clientId: string, limit: number) {
      const { data } = await this.supabase
        .from('content_feedback')
        .select('*')
        .eq('client_id', clientId)
        .order('created_at', { ascending: false })
        .limit(limit);

      return data || [];
    }

    async logFeedback(clientId: string, contentId: string, feedback: {
      liked: boolean;
      comments: string;
      tags: string[];
    }) {
      await this.supabase.from('content_feedback').insert({
        client_id: clientId,
        content_id: contentId,
        ...feedback,
        created_at: new Date().toISOString(),
      });
    }
  }

```



```
}  
  
export const clientStore = new ClientStore();
```

Step 3.2: Entry Point (src/index.ts)

```
import { messageBus } from './core/message-bus';  
import { routerAgent } from './agents/router';  
import { contextAgent } from './agents/context-agent';  
import { briefAgent } from './agents/brief-agent';  
import { draftAgent } from './agents/draft-agent';  
import { polishAgent } from './agents/polish-agent';  
import { reviewAgent } from './agents/review-agent';  
import { logger } from './utils/logger';  
  
async function main() {  
  logger.info('Starting Luniero Marketing Agent...');  
  
  // Connect to message bus  
  await messageBus.connect();  
  
  // Start all agents  
  await routerAgent.start();  
  await contextAgent.start();  
  await briefAgent.start();  
  await draftAgent.start();  
  await polishAgent.start();  
  await reviewAgent.start();  
  
  // Start consuming messages  
  const consumerName = `consumer-${process.pid}`;  
  messageBus.startConsuming(consumerName);  
  
  logger.info('Luniero Marketing Agent is running!');  
}  
  
main().catch((err) => {  
  logger.error('Failed to start', err);  
  process.exit(1);  
});
```

Step 3.3: CLI Interface (src/cli/index.ts)

```
import { routerAgent } from '../agents/router';  
import { stateStore } from '../core/state-store';  
import { messageBus } from '../core/message-bus';  
import { clientStore } from '../memory/client-store';  
  
async function main() {  
  const [, , command, ...args] = process.argv;  
  
  await messageBus.connect();  
  
  switch (command) {  
    case 'create-job': {  
      const [clientId, type, topic, platform] = args;  
      if (!clientId || !type || !topic) {
```

```

        console.log('Usage: create-job <clientId> <type> <topic>
[platform]');
        process.exit(1);
    }
    const jobId = await routerAgent.createJob({
        clientId,
        type: type as any,
        topic,
        platform,
    });
    console.log(`Job created: ${jobId}`);
    break;
}

case 'get-job': {
    const [jobId] = args;
    if (!jobId) {
        console.log('Usage: get-job <jobId>');
        process.exit(1);
    }
    const job = await stateStore.getJob(jobId);
    console.log(JSON.stringify(job, null, 2));
    break;
}

case 'setup-client': {
    const [clientId, name, industry] = args;
    if (!clientId || !name) {
        console.log('Usage: setup-client <clientId> <name>
[industry]');
        process.exit(1);
    }
    await clientStore.saveProfile(clientId, {
        id: clientId,
        name,
        industry: industry || 'general',
        description: '',
        goals: [],
        platforms: [],
        contacts: [],
        preferences: {
            contentPillars: [],
        },
    });
    console.log(`Client ${clientId} created`);
    break;
}

default:
    console.log(`
Luniero CLI

Commands:
    create-job <clientId> <type> <topic> [platform]    Create a new
content job
    get-job <jobId>                                     Get job status
and output
    setup-client <clientId> <name> [industry]          Create a client
profile
`);

```

```
    }

    await messageBus.disconnect();
  }

  main();
```

Phase 4: Database Setup

Supabase Tables (SQL)

```
-- Jobs table
CREATE TABLE jobs (
  id UUID PRIMARY KEY,
  client_id TEXT NOT NULL,
  type TEXT NOT NULL,
  status TEXT NOT NULL,
  input JSONB,
  context JSONB,
  research JSONB,
  brief JSONB,
  draft JSONB,
  polished_draft JSONB,
  review JSONB,
  output JSONB,
  iteration INTEGER DEFAULT 0,
  max_iterations INTEGER DEFAULT 3,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  completed_at TIMESTAMPTZ,
  error TEXT
);

-- Client profiles
CREATE TABLE client_profiles (
  id TEXT PRIMARY KEY,
  name TEXT NOT NULL,
  industry TEXT,
  description TEXT,
  goals JSONB,
  platforms JSONB,
  contacts JSONB,
  preferences JSONB,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Brand voices
CREATE TABLE brand_voices (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  client_id TEXT REFERENCES client_profiles(id),
  tone TEXT,
  avoid JSONB,
  examples JSONB,
  vocabulary JSONB,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```

-- Content feedback
CREATE TABLE content_feedback (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  client_id TEXT REFERENCES client_profiles(id),
  content_id TEXT,
  liked BOOLEAN,
  comments TEXT,
  tags JSONB,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_jobs_client ON jobs(client_id);
CREATE INDEX idx_jobs_status ON jobs(status);
CREATE INDEX idx_jobs_created ON jobs(created_at DESC);

```

Running the System

Step 1: Start Redis

```

# Using Docker
docker run -d --name redis -p 6379:6379 redis:alpine

# Or install locally
brew install redis && brew services start redis

```

Step 2: Set up environment

```

cp .env.example .env
# Edit .env with your API keys

```

Step 3: Run database migrations

```

# Run the SQL above in Supabase SQL editor

```

Step 4: Start the agent

```

npx tsx src/index.ts

```

Step 5: Create a test job

```

# In another terminal
npx tsx src/cli/index.ts setup-client acme "Acme Corp" "B2B SaaS"
npx tsx src/cli/index.ts create-job acme social_post "AI trends in
2026" linkedin

```

MVP Checklist

- ☐ Project structure created
- ☐ Redis running
- ☐ Supabase tables created
- ☐ Environment variables set
- ☐ Message bus working

- ☐ State store working
- ☐ Router agent working
- ☐ Context agent working
- ☐ Brief agent working
- ☐ Draft agent working
- ☐ Polish agent working
- ☐ Review agent working
- ☐ End-to-end job flow tested
- ☐ CLI working

Copy this entire document into Claude Code and say: "Let's build this step by step. Start with Phase 1."